

第二章

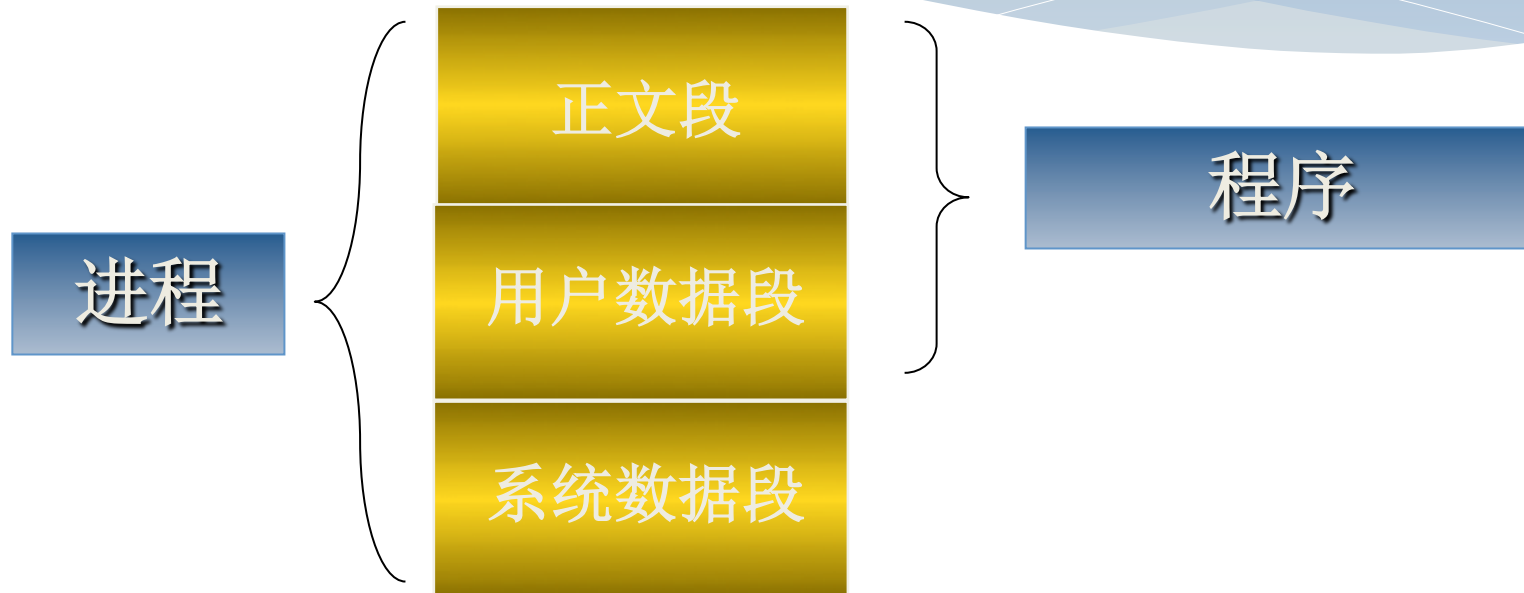
进程和线程

进程

进程的基本概念

- ◆ 进程是一个独立的可调度的任务
 - ◆ 进程是一个抽象实体。当系统在执行某个程序时，分配和释放的各种资源
- ◆ 进程是一个程序的一次执行的过程
- ◆ 进程和程序的区别
 - ◆ 程序是静态的，它是一些保存在磁盘上的指令的有序集合，没有任何执行的概念
 - ◆ 进程是一个动态的概念，它是程序执行的过程，包括创建、调度和消亡
- ◆ 进程是程序执行和资源管理的最小单位

进程和程序



进程实体

◆ 进程特性

- (1) 标识符：实际就是进程号，标识着这个进程
- (2) 状态：进程执行的状态
- (3) 程序计数器：记录程序运行的位置，称为pc指针，如果没有pc指针，当程序执行到时间片结束，再次运行程序又必须从开始重新运行
- (4) 优先级：类似于银行里面的VIP用户
- (5) I/O资源信息:进程打开的文件资源等

查看系统中的进程

- ◆ **ps**指令：通常可以查看到：进程的ID、进程的用户ID、进程状态和进程的动作command

```
[root@localhost other]# ps -aux
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.0	0.1	2048	632	?	Ss	Dec09	0:11	init [3]
root	2	0.0	0.0	0	0	?	S	Dec09	0:00	[migration/0]
root	3	0.0	0.0	0	0	?	SN	Dec09	0:00	[ksoftirqd/0]
root	4	0.0	0.0	0	0	?	S	Dec09	0:01	[watchdog/0]
root	5	0.0	0.0	0	0	?	S<	Dec09	0:07	[events/0]
root	6	0.0	0.0	0	0	?	S<	Dec09	0:00	[khelper]
root	7	0.0	0.0	0	0	?	S<	Dec09	0:00	[kthread]
...	...	...	...	...	...	...	...	...	...	...
root	3093	0.0	0.0	1640	432	tty4	Ss+	Dec09	0:00	/sbin/mingetty tty4
root	3094	0.0	0.0	1640	428	tty5	Ss+	Dec09	0:00	/sbin/mingetty tty5
root	3112	0.0	0.0	1636	452	tty6	Ss+	Dec09	0:00	/sbin/mingetty tty6

查看系统中的进程

* ps常用参数

ps -A:显示当前系统所有进程的信息，如同-e

ps -aux:显示所有进程所有属性值的信息，

ps -A -o pid,ppid,command 输出所有进程信息里面具体字段的信息状态

* 常见属性的意义:

user:用户

pid:进程号

%cpu:cpu所占率

%mem:内存所占率

tty:所依赖的终端

stat:进程的状态

command:执行的是什么动作

查看系统中的进程

- ◆ STAT 进程状态
 - ◆ R 正在运行可中在队列中可过行的；
 - ◆ S 处于休眠状态；
 - ◆ T 停止或被追踪；
 - ◆ Z 僵尸进程；
 - ◆ < 优先级高的进程
 - ◆ N 优先级较低的进程
 - ◆ L 有些页被锁进内存；
 - ◆ s 进程的领导者（在它之下有子进程）；
 - ◆ + 位于后台的进程组；

查看进程树

- ◆ `pstree` 查看进程树

- ◆ 默认查看当前系统所有的进程

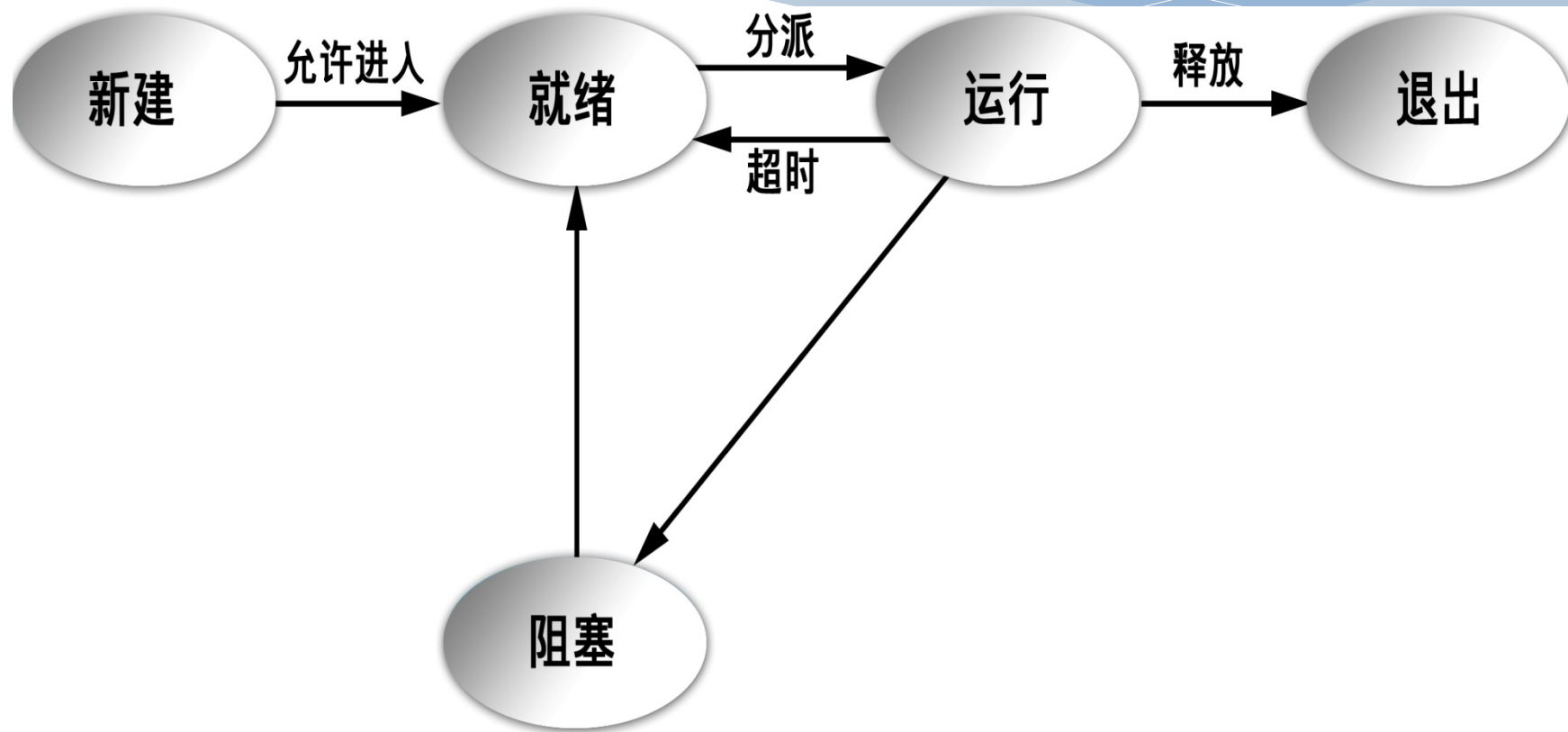
- ◆ `pstree -p`

- ◆ 查看当前系统所有的进程以及显示进程号

- ◆ `pstree -p 进程号pid`

- ◆ 显示具体进程下面的子进程

进程状态



进程ID

- ◆ 进程ID定义:

- ◆ 每个进程都有一个独一无二的进程号用于区别其他的进程, 简称pid, process id
- ◆ 每个进程都由父亲进程创建, 所以每个进程都有一个父亲进程, 除了init进程, inti进程是最初的祖宗进程

获得进程id号

函数原型	pid_t getpid(void): 要获得当前进程的ID pid_t getppid(void): 获得当前进程的父进程的ID
头文件	#include <unistd.h> #include <sys/types.h>
参数	无参
返回值	看函数的功能

system创建进程

函数原型	<code>int system(const char *);</code>
头文件	<code>#include <stdlib.h></code>

利用system函数调用程序结果与从shell中执行这个程序一样，事实上system建立了一个shell的子进程，system依赖于shell，这种方法不是通用的，当我们建立一个程序，我们不应该依赖于任何其他的东西

system是一个阻塞函数，创建出来的子进程结束后，system才返回

示例代码

```
int main()
{
    printf("pid=%d\n",getpid());
    while(1);
    return 0;
}
```

```
int main()
{
    printf("pid=%d\n",getpid());
    system("./1");

    printf("====\n");
    return 0;
}
```

exec族 替换进程

函数原型	<pre>int execl(const char *path, const char *argo,...); int execv(const char *path, char *const argv[]);</pre>
头文件	<pre>#include <unistd.h></pre>

exec函数族提供了一种在进程中启动另一个程序执行的方法。它可以根据指定的文件名或目录名找到可执行文件，并用它来取代原调用进程的数据段、代码段和堆栈段。在执行完之后，原调用进程的内容除了进程号外，其他全部都被替换了。

示例代码 : execl

```
int main(int argc, char *argv[])
{
    printf("pid=%d\n", getpid());
    printf("%s--%s\n", argv[1], argv[2]);
    return 0;
}
```

```
int main()
{
    printf("pid=%d\n", getpid());
    execl("./1", "1", "第一个参数", "第二个参数", NULL);
    printf("end\n");
    return 0;
}
```


示例代码 :execv

```
int main(int argc,char *argv[])
{
    printf("pid=%d\n",getpid());
    printf("%s--%s\n",argv[1],argv[2]);
    return 0;
}
```

```
int main()
{
    printf("pid=%d\n",getpid());
    char *argv[]={"1","第一个参数","第二个参数",NULL};
    execv("./1",argv);
    printf("end\n");
    return 0;
}
```

fork派生进程

函数原型	<code>pid_t fork();</code>
头文件	<code>#include<unistd.h></code> <code>#include<sys/types.h></code>

- ◆通过复制父进程在内存中的数据，创建父进程的副本：
 - ◆在父进程中，**fork**返回新创建子进程的进程**PID**。
 - ◆在子进程中，**fork**返回**0**。
 - ◆如果出现错误，**fork**返回**-1**。

思考

思考：如下输出

```
#include <unistd.h>
int main()
{
    pid_t pid;
    pid=fork();
    printf("=====\n");
    return 0;
}
```

思考：怎么让父子进程做不同的事情？

思考

思考：如下输出

```
#include <unistd.h>
int main()
{
    int n=0;
    pid_t pid;
    pid=fork();
    if(pid==0)
    {
        n=3;
    }
    else
    {
        sleep(3);
        printf("n=%d\n",n);
    }
    return 0;
}
```

思考

思考：如下输出

```
#include <unistd.h>
int main()
{
    int i=0;
    for(i=0;i<3;i++)
    {
        pid_t fpid=fork();
        if(fpid==0)
        {
            printf("son\n");
        }
        else
        {
            printf("father\n");
        }
    }
    return 0;
}
```

僵尸进程

◆僵尸进程的产生?

- (1) 一个进程结束之后会有一些残留的资源，如果不去回收释放，那么死去的尸体就一直占用着内存
- (2) 父进程没有结束，子进程已经提前结束了
- (3) 子进程结束会发送SIGCHLD给父进程
- (4) 当父进程如果结束了，那么僵尸进程就不会存在了，因为init进程会最终接管僵尸进程，为他释放资源

总结：僵尸进程存在于父亲进程没有结束，自己先结束，然而父亲进程没有为子进程做任何的处理事务

示例代码

```
int main()
{
    pid_t pid;
    pid = fork();
    if (pid < 0)
    {
        perror("fork error:");
        exit(1);
    }
    else if (pid == 0)
    {
        printf("子进程id=%d\n", getpid());
        exit(0);
    }
}
```

示例代码

```
else if(pid>0)
{

    sleep(1000);

}
return 0;
}
```

使用ps -A -o pid,stat,command

不处理僵尸进程的后果

- ◆ 若子进程结束后，称之为僵尸进程，一般情况下，会有相应的进程立即(通常是父进程)，对僵尸进程进行善后处理，即销毁其pcb,回收pid。
- ◆ 若没有相应进程对其进行处理，则该僵尸进程一直存在，占用pid，pcb块不会销毁，若系统存在大量的僵尸进程，那么创建新进程的时候，可能会由于没有pid 或者其他资源而导致失败。

wait等待进程

函数原型：pid_t wait(int *status);

头文件：#include <unistd.h>

参数：存储子进程退出的状态码的地址，回收进程的资源

返回值：失败<0,成功返回子进程ID

如果子进程在父进程调用wait之前结束，子进程会变成一个僵尸进程，当父进程调用wait，僵尸进程的结束状态被提取出来，子进程所占用的资源被回收，并且wait函数立即返回

常用的宏

◆-WIFEXITED(status):

- ◆这个宏用来指出子进程是否为正常退出的，如果是，它会返回一个非零值

◆-WEXITSTATUS(status):

- ◆当WIFEXITED返回非零值时，我们可以用这个宏来提取子进程的返回值，如果子进程调用exit(5)退出，WEXITSTATUS(status)就会返回5；如果子进程调用exit(7)，WEXITSTATUS(status)就会返回7。请注意，如果进程不是正常退出的，也就是说，WIFEXITED返回0，这个值就毫无意义

示例代码 1 of 2

```
int main()
{
    pid_t pid;
    pid=fork();
    if(pid==0)
    {
        //abort();//异常退出
        return 3;
    }
    else if(pid>0)
    {
        int status;
        wait(&status);
    }
}
```

示例代码 2 of 2

```
if(WIFEXITED(status))
{
    printf("退出码=%d\n",WEXITSTATUS(status));
}

}
else
{
    printf("fork error\n");
    return 0;
}

}
```

waitpid等待进程

函数原型：pid_t waitpid(pid_t pid, int *, int);

参数：

参数一：等待退出的子进程ID号

参数二：子进程退出的状态码，不关心可以置NULL

参数三：

WNOHANG：如果没有任何已经结束的子进程则马上返回，不予以等待。

WUNTRACED：如果子进程进入暂停执行情况则马上返回，但结束状态不予以理会。

0：不想使用上面的选项，那么效果就和wait一样，是一个阻塞的函

返回值：

当正常返回的时候，waitpid返回收集到的子进程的进程ID；

如果设置了选项WNOHANG，而调用中waitpid发现没有已退出的子进程可收集，则返回0；

如果调用中出错，则返回-1，这时errno会被设置成相应的值以指示错误所在；

示例代码 1 of 3

```
int main()
{
    int status;
    pid_t pid;
    pid_t pid_c;
    pid=fork();
    if(pid==0)
    {
        sleep(5);
        exit(4);
    }
}
```

示例代码 2 of 3

```
else if(pid>0)
{
    do
    {
        pid_c=waitpid(pid,&status,WNOHANG);
        if(pid_c==0)
        {
            printf("Not child proess exit\n");
            sleep(1);
        }
    }while(pid_c==0);
```


示例代码 3 of 3

```
if(WIFEXITED(status))
{
    printf("the child proess %d exit normally.\n",pid_c);
    printf("the return code is %d.\n",WEXITSTATUS(status));
}
else
{
    printf("the child process %d exit abnormally\n",pid_c);
}
}
```

思考

- 思考：处理僵尸进程的时候，父进程会等待子进程的结束，或者循环试探性的等待子进程的结束？这样父进程就没有办法干别的事情？如果父进程很忙？那么子进程应该怎么处理？

解决思路：让僵尸进程交给别的进程操作。让子进程产生一个孙子进程，让子进程退出，孙子进程就归init进程所管辖范围之内，init进程会释放孙子进程的资源，原来子进程所做的事情交给孙子进程处理

示例代码 1 of 2

```
int main()
{
    pid_t pid=fork();
    if(pid==0)
    {
        pid=fork();
        if(pid>0)
            exit(0);
        else if(pid==0)
        {
            sleep(2);
            printf("pid=%d---ppid=%d\n",getpid(),getppid());
        }
    }
}
```

示例代码 2 of 2

```
else if(pid>0)
{
    int status;
    wait(&status);
    while(1)
    {
        printf("我在忙...\n");
        sleep(1);
    }
}
```

进程的启动

```
int main(int argc, char *argv[])  
{  
  
  
}
```

实际上一个程序启动需要fork+exec

比如：终端启动一个程序，本身终端就是一个程序，在终端内部启动子程序，使用fork，然而子进程还不能走原来的父进程（终端那样运行），所以要用exec替换成我们现在要运行的程序

进程的终止

◆ 5种正常终止

main函数return

_Exit/_exit #include<unistd.h>

exit #include<stdlib.h>

进程最后一个线程return

最后一个线程pthread_exit

◆ 异常终止

调用abort

接到一个信号终止

最后一个线程对取消请求做出响应

注册终止函数

终止函数：main函数结束之前调用的函数

```
void exit_fn1(void)
{
    printf("%s\n", __func__);
}
void exit_fn2(void)
{
    printf("%s\n", __func__);
}
int main(void)
{
    atexit(exit_fn1);
    atexit(exit_fn2);
}
```

注册终止函数注意点

- ◆ 先注册的函数后调用
- ◆ 异常终止和 `_exit` 和 `_Exit` 不会调用注册终止函数
- ◆ 终止函数的用途：
 - ◆ 在程序结束后归还空间
 - ◆ 在程序结束前，关闭所有的文件描述符

exit、_exit和return

◆ exit和return的区别?

exit结束后是将控制权交给系统;

return结束是函数的返回, 返回完成后将控制权交给系统

◆ exit和_exit函数的区别?

_exit执行后立即返回给内核, 调用_exit会将文件描述符关闭, 并做其他的一些处理方式, 但是不会刷新流(输入流, 输出流。。。)

exit是对_exit的一层封装, 会刷新流, 底层会调用_exit

孤儿进程

- ◆ 定义：父进程退出，子进程还存在，孤儿进程的存在是一瞬间的，系统一发现孤儿进程，该孤儿进程就会被进程号为1的init进程所收养。

守护进程

◆守护进程的特征

- (1) 守护进程没有控制终端
- (2) 无法在屏幕上打印信息也无法接收键盘输入
- (3) 后台运行
- (4) 常常在系统自举时启动，在系统关闭时才终止
- (5) 大多数守护进程的父进程是init进程

创建守护进程的步骤

- (1) 创建子进程，父进程退出
- (2) 在子进程中创建新的会话
- (3) 改变当前目录为根目录
- (4) 重设文件掩码
- (5) 关闭文件描述符

创建子进程，父进程退出

◆ 第一步完成以后，子进程就在形式上做到了与控制终端的脱离

◆ 由于父进程已经先于子进程退出，子进程变成孤儿进程

```
pid = fork();
```

```
if (pid > 0) /*父进程退出*/  
{  
    exit(0);  
}
```

在子进程中创建新的会话

◆ 进程组

进程组是一个或多个进程的集合。进程组由进程组ID来唯一标识。

每个进程组都有一个组长进程，进程组ID就是组长进程的进程号。

◆ 会话期

会话组是一个或多个进程组的集合

setsid创建新的会话

◆ setsid函数作用

setsid函数用于创建一个新的会话

◆ 为什么要这么做？

创建守护进程的第一步调用了fork函数来创建子进程，再将父进程退出。由于在调用了fork函数时，子进程全盘拷贝了父进程的会话期、控制终端等，虽然父进程退出了，但会话期、进程组、控制终端等并没有改变，因此，这还不是真正意义上的独立开来

改变当前目录为根目录

- ◆ 通常的做法是让“/”或“/tmp”作为守护进程的当前工作目录。
- ◆ 在进程运行过程中，当前目录所在的文件系统是不能卸载的。
- ◆ chdir函数可以改变进程当前工作目录

重设文件权限掩码

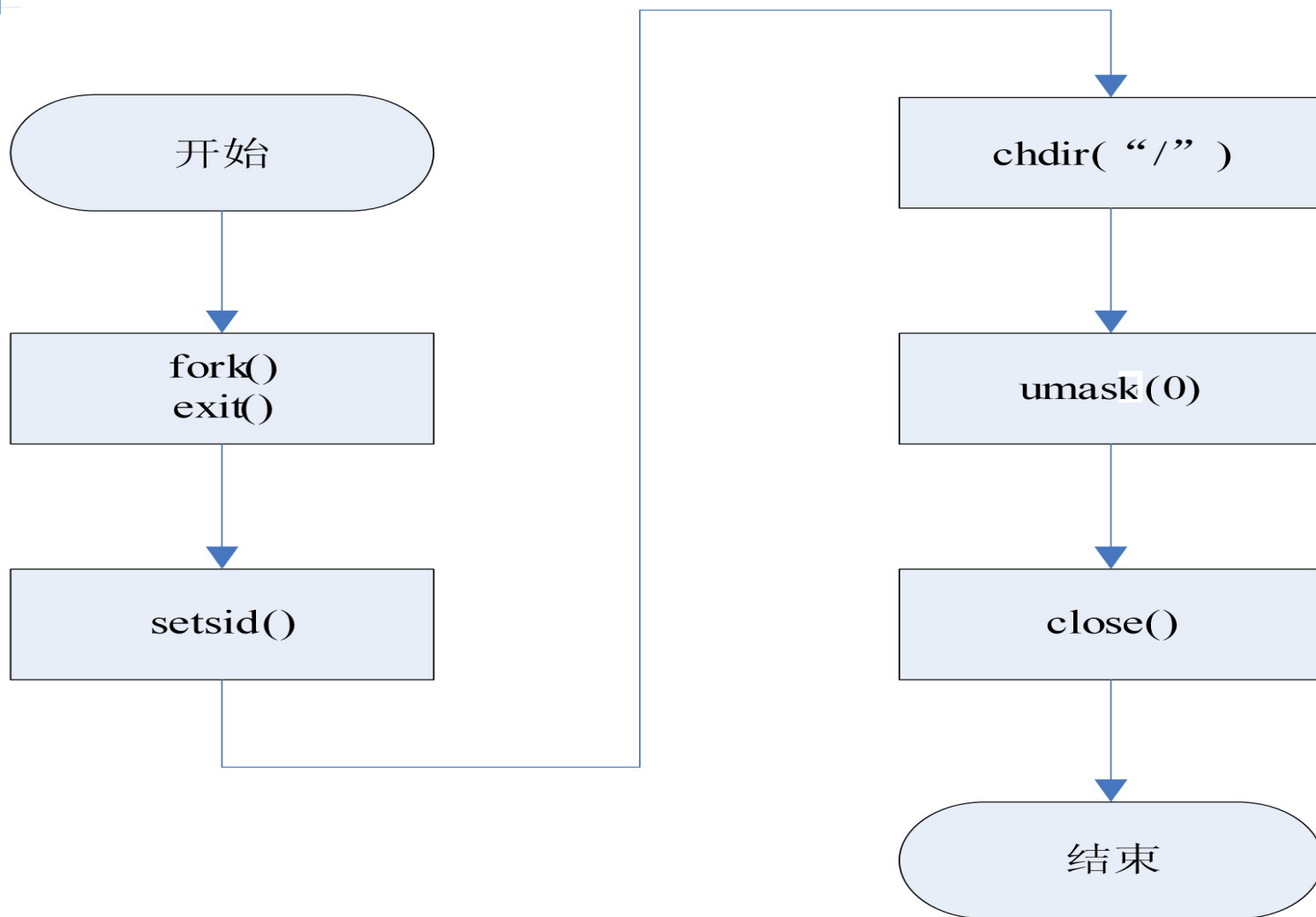
- ◆ 文件权限掩码是指文件权限中被屏蔽掉的对应位。把文件权限掩码设置为0，可以增加该守护进程的灵活性。
- ◆ 设置文件权限掩码的函数是umask
- ◆ 通常的使用方法为umask(0)

关闭文件描述符

- ◆新建的子进程会从父进程那里继承所有已经打开的文件。
- ◆在创建完新的会话后，守护进程已经脱离任何控制终端，应当关闭用不到的文件

```
for (i=0; i<MAXFILE; i++)  
{  
    close(i);  
}
```

守护进程创建流程



练习

◆ 练习：实现往文件里面写内容，使用守护进程